

MAP-REDUCE

The programming paradigm for Big Data

Real MapReduce exercise

Exercise setup

Subdivide yourselves into groups

- Each group is a cluster
- Each person is a machine!

Background

- We have a big datasets about e-commerce sales
- Data is spread in blocks across the "machines"

Goal

- Collaborate to answer a simple analytical question

MapReduce algorithms

Disambiguation of MapReduce

"MapReduce is a programming model and an associated implementation for processing and generating large data sets.

Users specify a map function that processes a key/value pair to generate a set of intermediate key/value pairs, and a reduce function that merges all intermediate values associated with the same intermediate key."

-- Dean J., Ghemawat S. (Google)

Hadoop MapReduce is an open-source implementation of the MapReduce programming model

The goal

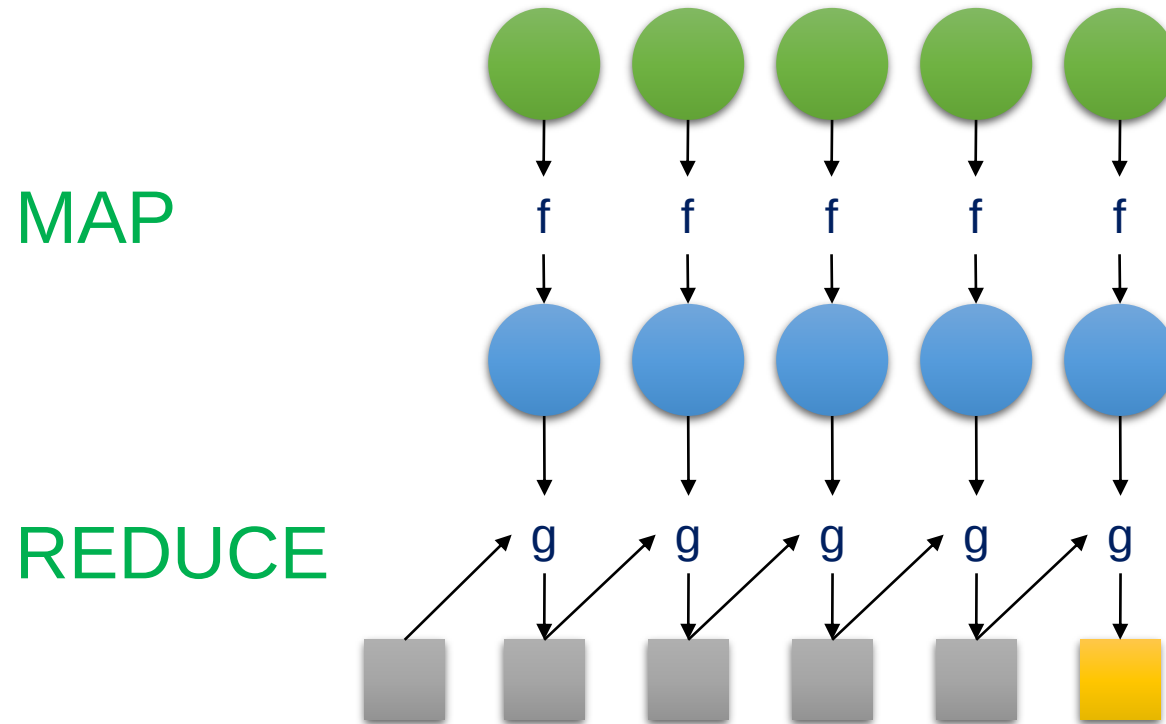
MapReduce aims to support analytical jobs over large datasets

- Iterate over a large number of records (e.g., a dataset of customer)
- Extract something of interest from each (e.g., their department and salary)
- Aggregate results (e.g., compute the average salary by department)
- Save results to the DFS

The functions

MAP: apply a function f to every element in a list

REDUCE: iteratively apply a function g to aggregate elements



Roots in Functional Programming

Imperative programming

```
a = 0
b = a + 1
```

```
### Map example to obtain [4,4,3]
names = ['Mary', 'Isla', 'Sam']
name_lengths = []
for i in range(len(names)):
    name_lengths[i] = len(names[i])

### Reduce example to obtain 3
sentences = [
    'Mary read a story to Sam and Isla.',
    'Isla cuddled Sam.', 'Sam chortled.' ]
sam_count = 0
for sentence in sentences:
    sam_count += sentence.count('Sam')
```

Functional programming

```
a = 0
b = increment(a)
def increment(a):
    return a + 1;
```

```
### Map example to obtain [4,4,3]
names = ['Mary', 'Isla', 'Sam']
name_lengths = map(len, names)

### Reduce example to obtain 3
sentences = [
    'Mary read a story to Sam and Isla.',
    'Isla cuddled Sam.', 'Sam chortled.' ]
sam_count = reduce(
    lambda a, x: a + x.count('Sam'),
    sentences, 0
)
```

Use case – Travel agency

A travel agency offers hotel rooms on Trivago, competing with other agencies to offer the best value for money

- For a given user search (e.g., 4 nights in Rome for 2 people), 25 offers are returned
- Every day, 40M user searches are made → 1B offers are returned
- A log of every offer is maintained; 1KB per offer = **1TB per day**
 - Info about hotel and room type + a lot of statistics

Hotel	City	Country	Date	TTT	LOS	GS	Market	Price	Markup	CMP	Impressions	Clicks	Bookings
The Majestic	Rome	Italy	22/09/24	4	2	1	EU	150	5	155	23	10	2
Augustian	Rome	Italy	22/09/24	4	2	1	EU	160	10	140	16	5	0
Cosmopolitan	Milan	Italy	19/12/24	10	1	2	NA	120	20	110	42	11	0
The Empire	NYC	USA	31/12/24	30	5	3	AS	130	2	125	8	1	1
Presidential	Paris	France	28/10/24	1	1	2	EU	150	3	140	4	2	1
Royal Orchid	London	UK	06/01/25	45	7	4	EU	100	-5	100	15	10	1

Legend: TTT = time to travel (in number of days); LOS = length of stay (in number of days); GS = group size; CMP = cheapest market price

Use case – Travel agency

Queries

- In which countries (or markets) do I make the most revenues (i.e., markup)?
- What is the rate of converting impressions into clicks? And clicks into bookings?
- In which combinations of City-TTT-LOS-GS am I most competitive?
- Over which hotels am I never competitive?

Hotel	City	Country	Date	TTT	LOS	GS	Market	Price	Markup	CMP	Impressions	Clicks	Bookings
The Majestic	Rome	Italy	22/09/24	4	2	1	EU	150	5	155	23	10	2
Augustian	Rome	Italy	22/09/24	4	2	1	EU	160	10	140	16	5	0
Cosmopolitan	Milan	Italy	19/12/24	10	1	2	NA	120	20	110	42	11	0
The Empire	NYC	USA	31/12/24	30	5	3	AS	130	2	125	8	1	1
Presidential	Paris	France	28/10/24	1	1	2	EU	150	3	140	4	2	1
Royal Orchid	London	UK	06/01/25	45	7	4	EU	100	-5	100	15	10	1

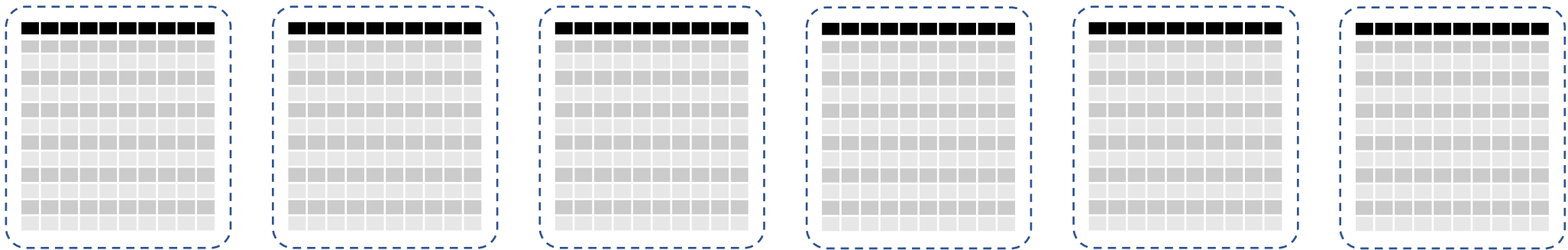
Legend: TTT = time to travel (in number of days); LOS = length of stay (in number of days); GS = group size; CMP = cheapest market price

Use case – Travel agency

Queries

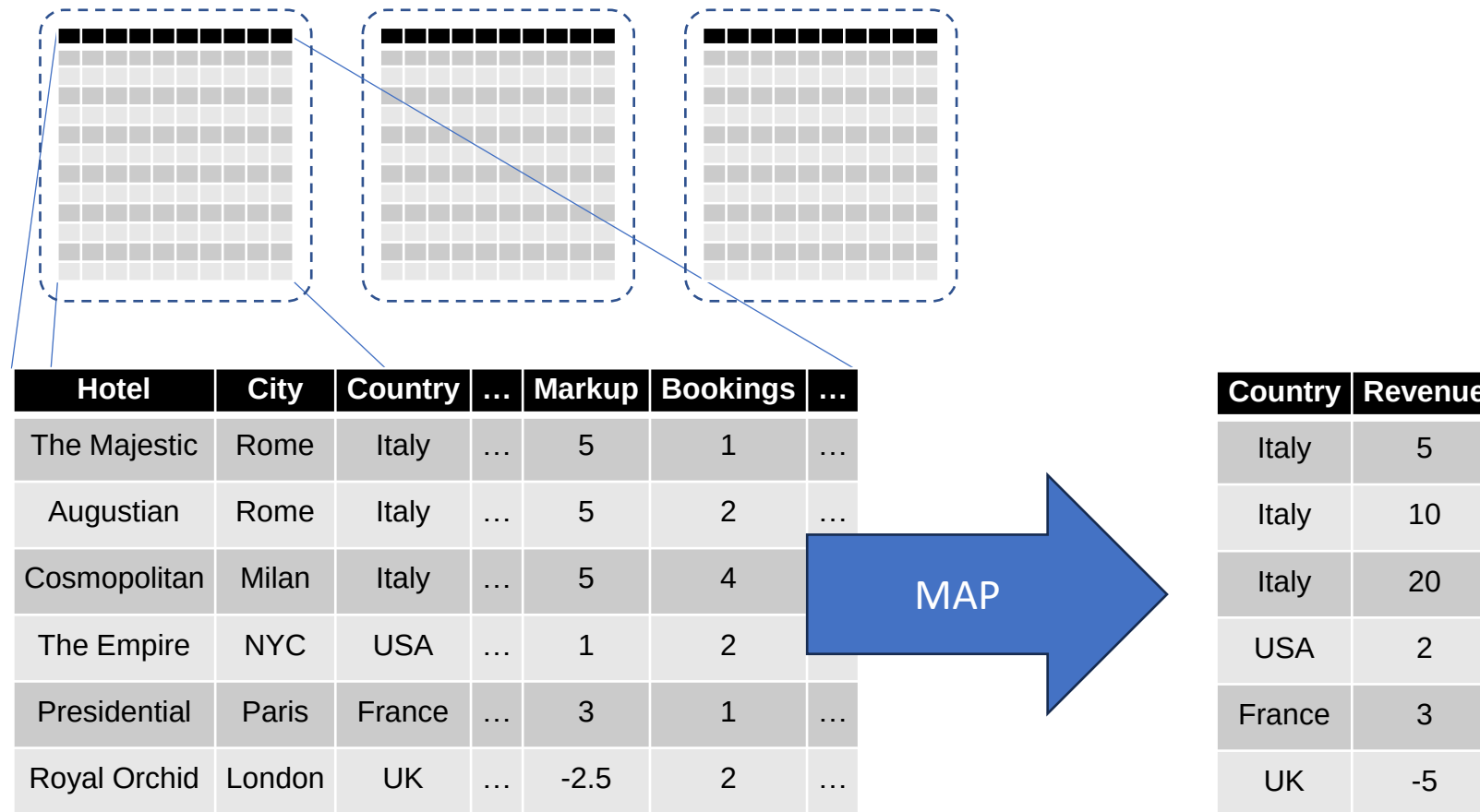
- In which countries (or markets) do I make the most revenues (i.e., markup)?
- What is the rate of converting impressions into clicks? And clicks into bookings?
- In which combinations of City-TTT-LOS-GS am I most competitive?
- Over which hotels am I never competitive?

How do we calculate them, considering that the data is distributed over multiple nodes in the cluster?



MapReduce

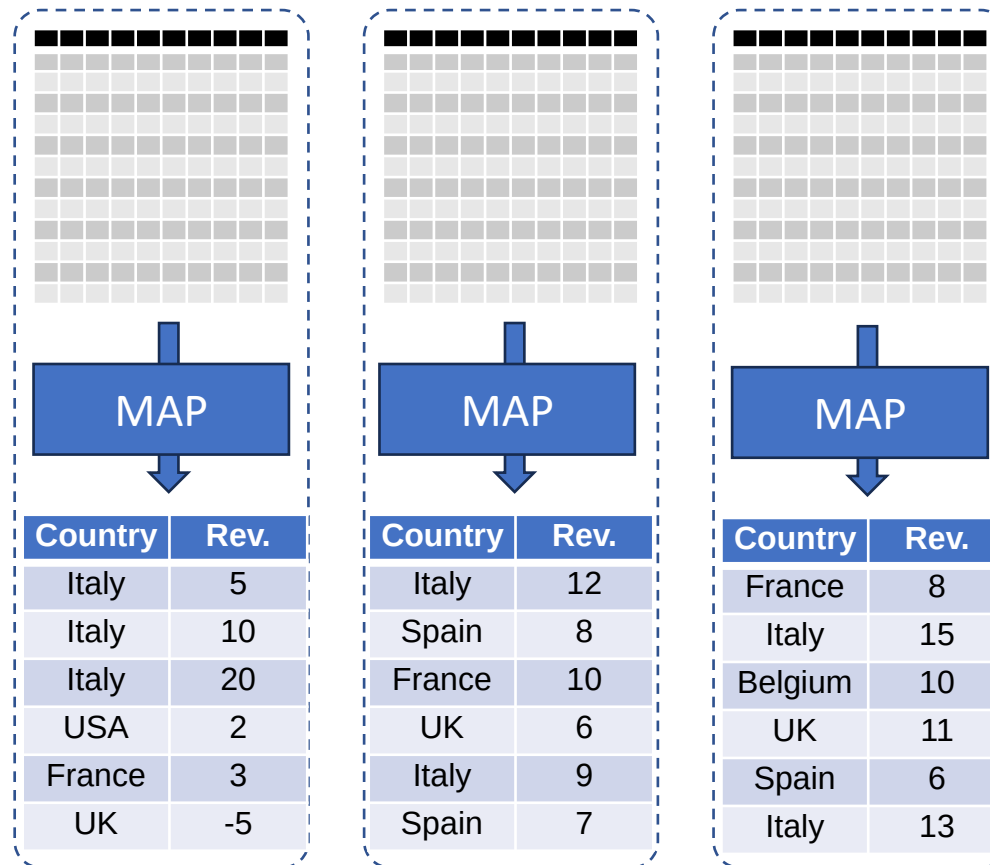
How much revenue (i.e., markup * #bookings) is made in each country?



MapReduce

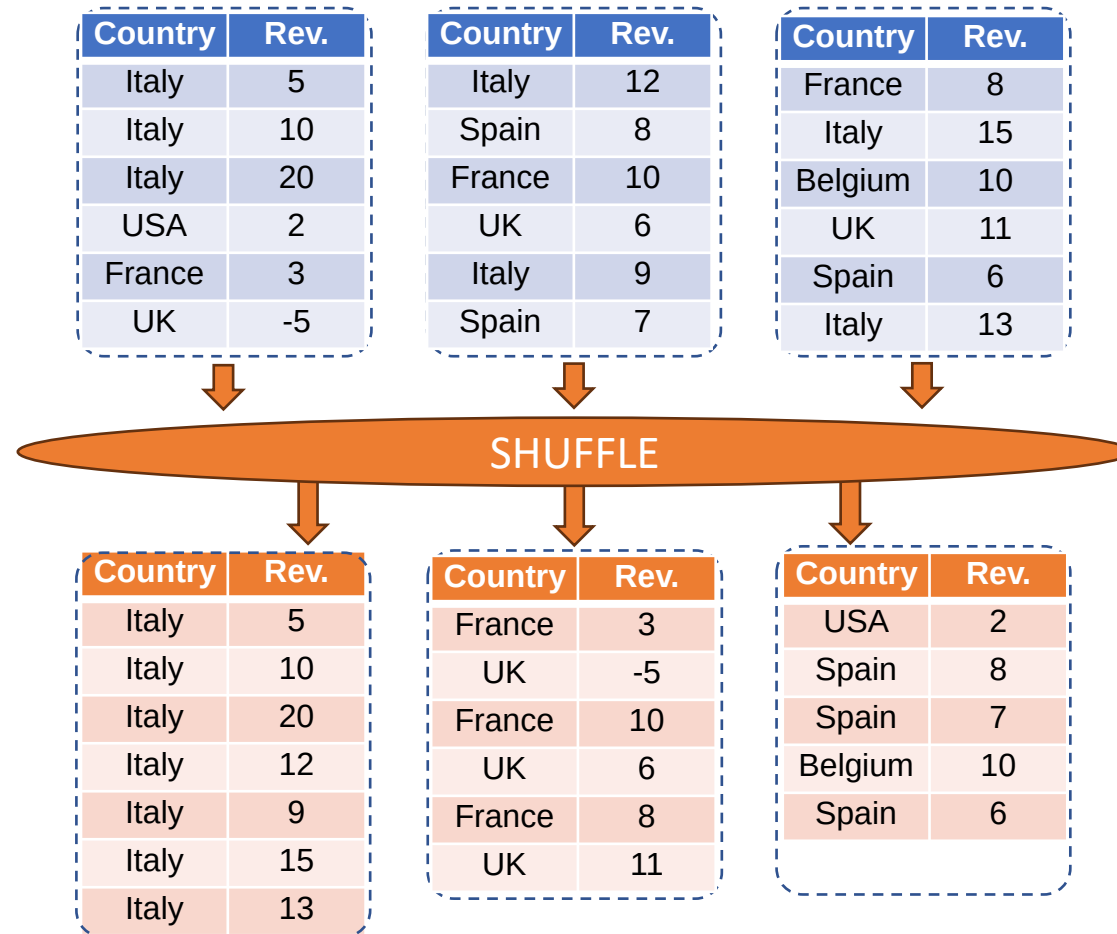
Map tasks (i.e., processes) locally extract the needed data

- What then?



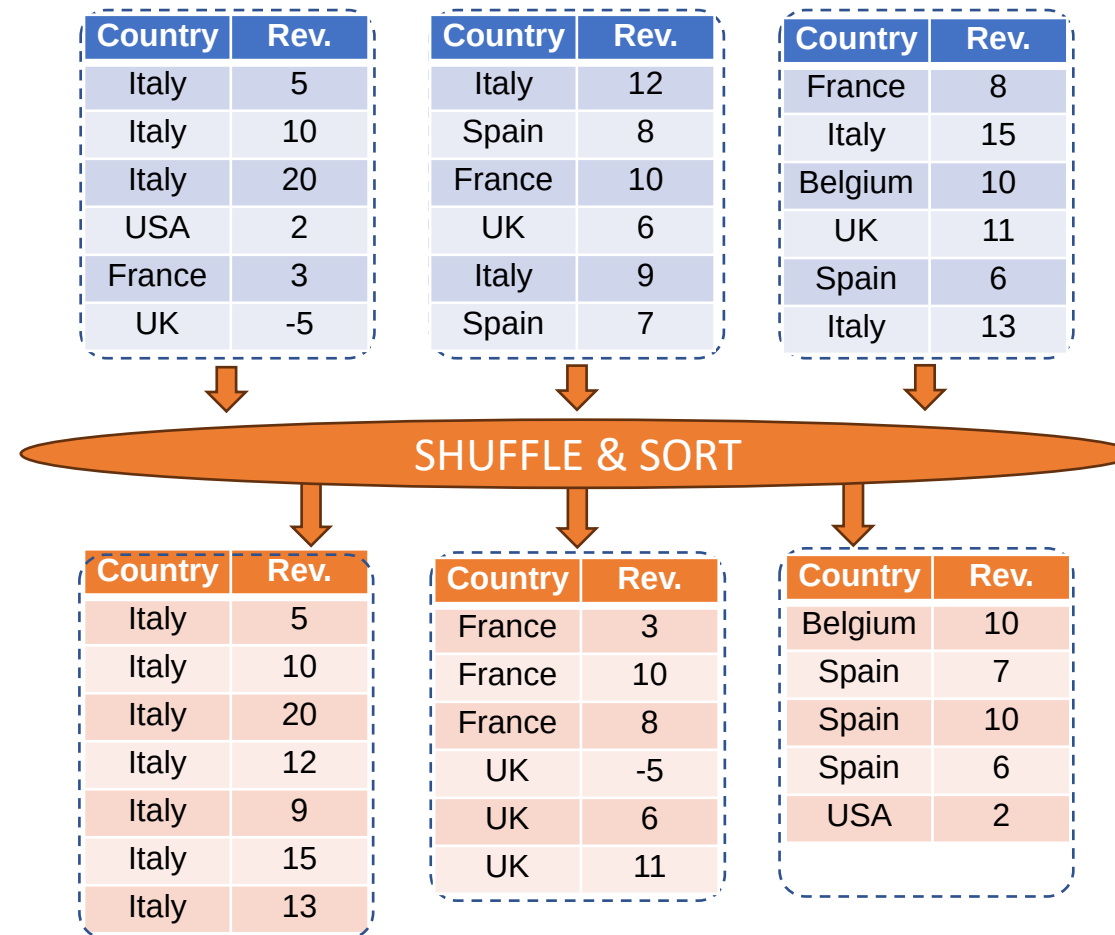
MapReduce

Shuffling is a global operation to redistribute data, on the basis of how it is meant to be aggregated



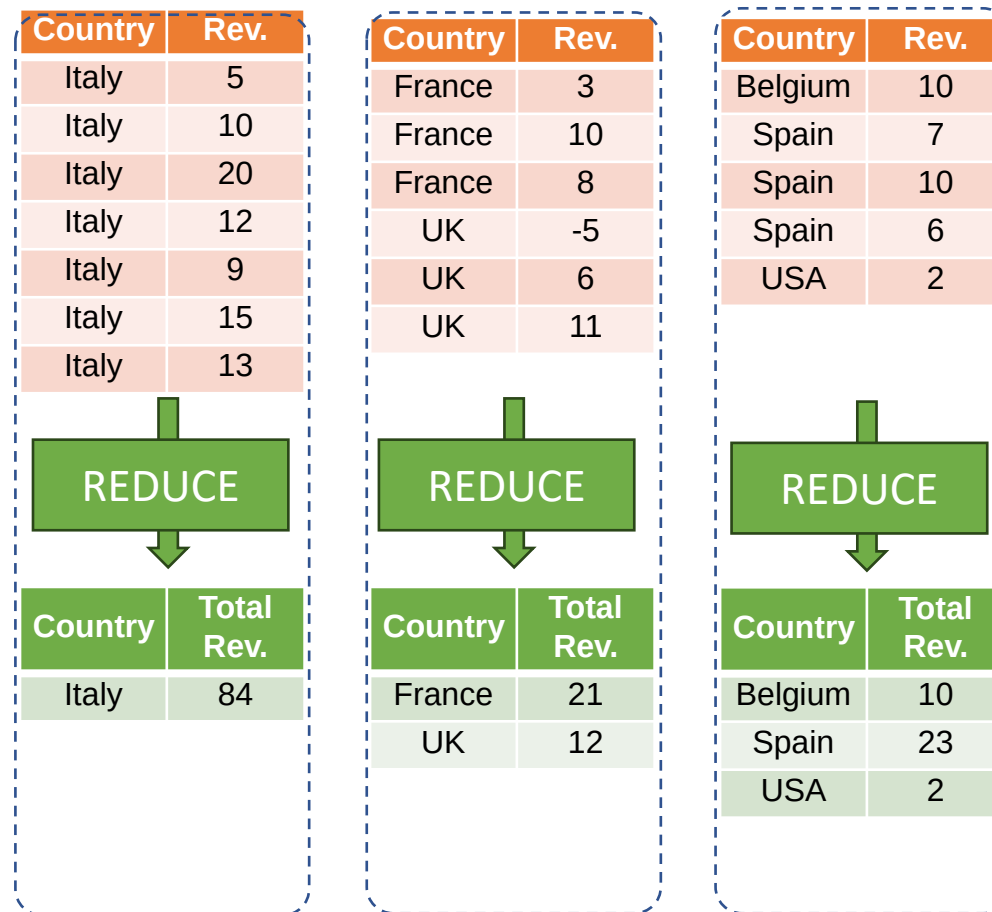
MapReduce

It is actually *Shuffle & Sort*, as sorting is locally done to enable the later aggregation



MapReduce

Reduce tasks (i.e., processes) locally perform the aggregation



MapReduce

The **map operation** defines what must be extracted from the data, in the form of **key-value pairs**

- It can be parallelized in a straightforward manner, since each functional application happens in isolation
- The type of key-value pair can be chosen by the programmer

The **reduce operation** defines how the values corresponding to the same keys must be aggregated/combined

- To be applied, all values corresponding to the same key must be “brought together” on the same machine

The programmer defines the map and the reduce functions

The execution framework handles everything else!

MapReduce process

1) Input is divided into fixed-size *splits*

- Smaller splits → faster processing, better load-balancing; but increased management overhead
- Optimal split size = DFS block size

2) A *Map task* is created for each split

- The task runs the user-defined map function for each *record* in the split

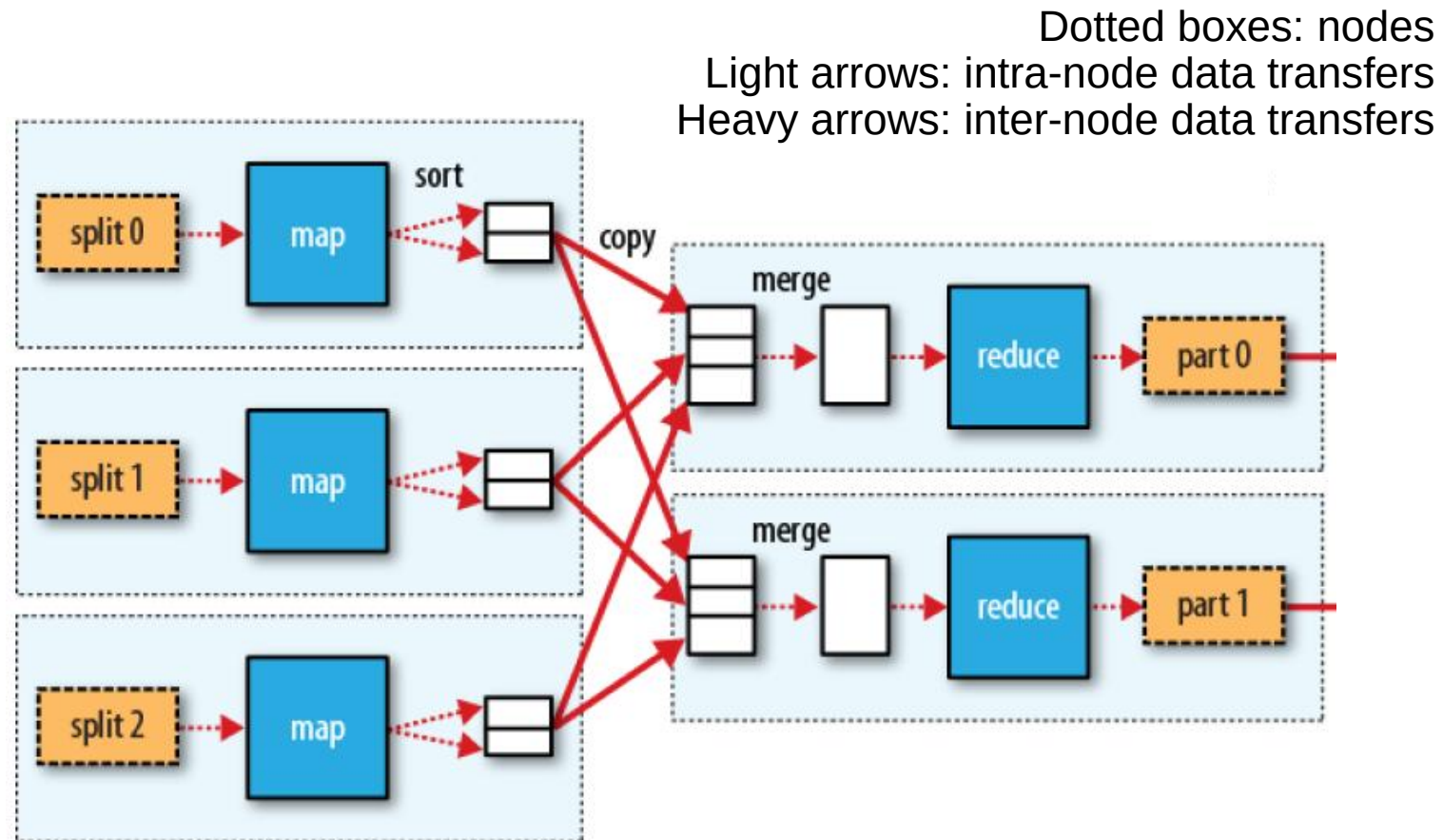
3) The key-value pairs returned by Map tasks are sorted and **stored in the local disk**

- Map output is intermediate output; storing it in HDFS with replication would be "overkill"
- Map output is **sorted by key** and subdivided into partitions - one for each reducer

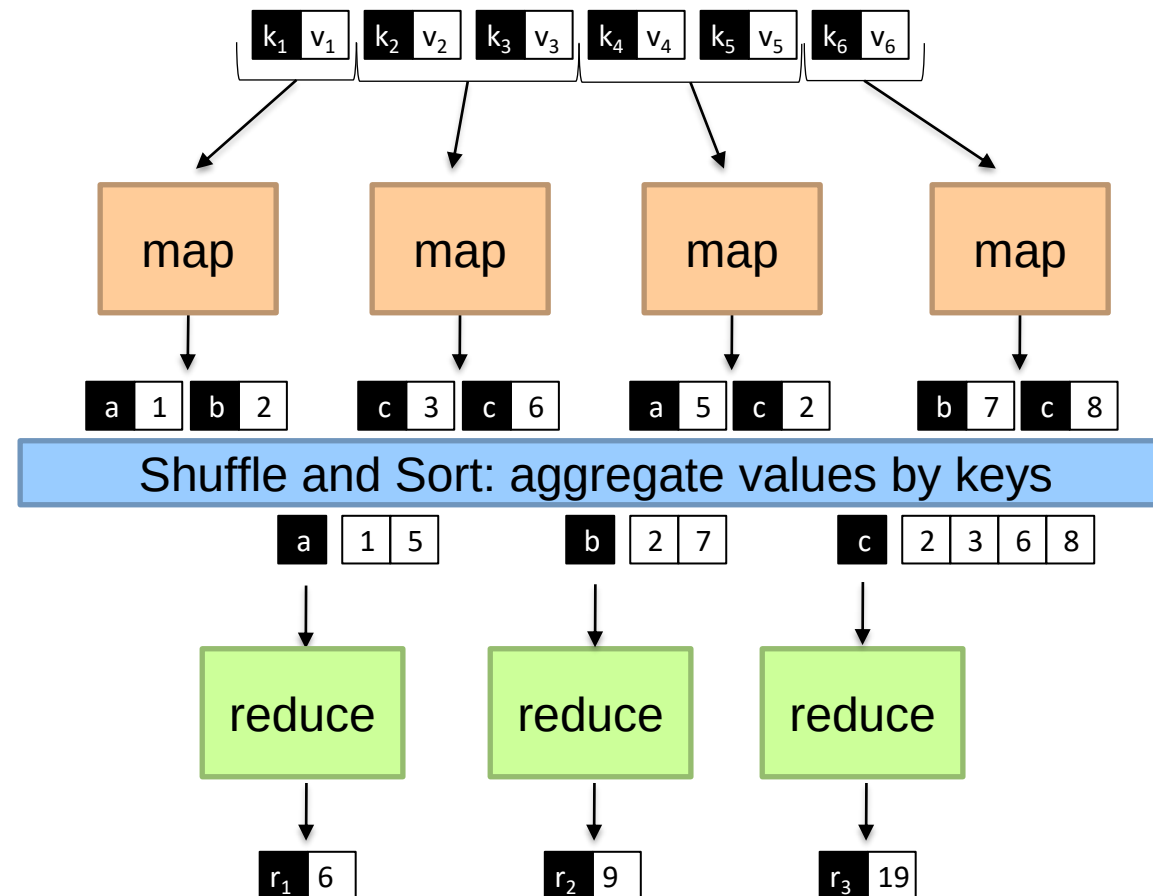
4) Map outputs are sent to the nodes where the *Reduce tasks* are running

- The task runs the user-defined reduce function to combine the values of a given key
- **The number of Reduce task can be specified independently**
- Inputs from different mappers are **merged and sorted by key** before running the reduce function
- The values for a given key fall to the same reducer; a reducer may handle more keys (one at a time)

MapReduce process



MapReduce process: an example



Example: Word Count

Problem: counting the number of occurrences for each word in a collection of documents

Input: a repository of documents; each document is a value in the input pairs

Map function: read a document and emit a sequence of key-value pairs

- Keys are the words found in the documents; values are equal to 1:

$(w_1, 1), (w_2, 1), (w_1, 1), \dots, (w_n, 1)$

Shuffle and Sort: group by key and generate pairs of the form:

$(w_1, [1, 1, \dots, 1]), \dots, (w_n, [1, 1, \dots, 1])$

Reduce function: add up all the values for a given key and emits a pair of the form:

(w, m)

Output: w is a word that appears at least once among all the input documents; m is the total number of occurrences of w among all those documents

Another example: Word Length Count

Problem: counting how many words of certain lengths exist in a collection of documents

Input: a repository of documents ; each document is a value in the input pairs

Map function: read a document and emit a sequence of key-value pairs

- The key is the length of a word; the value is the word itself:

$$(i, w_1), \dots, (j, w_n)$$

Shuffle and Sort: group by key and generate pairs of the form:

$$(1, [w_1, \dots, w_k]), \dots, (n, [w_r, \dots, w_s])$$

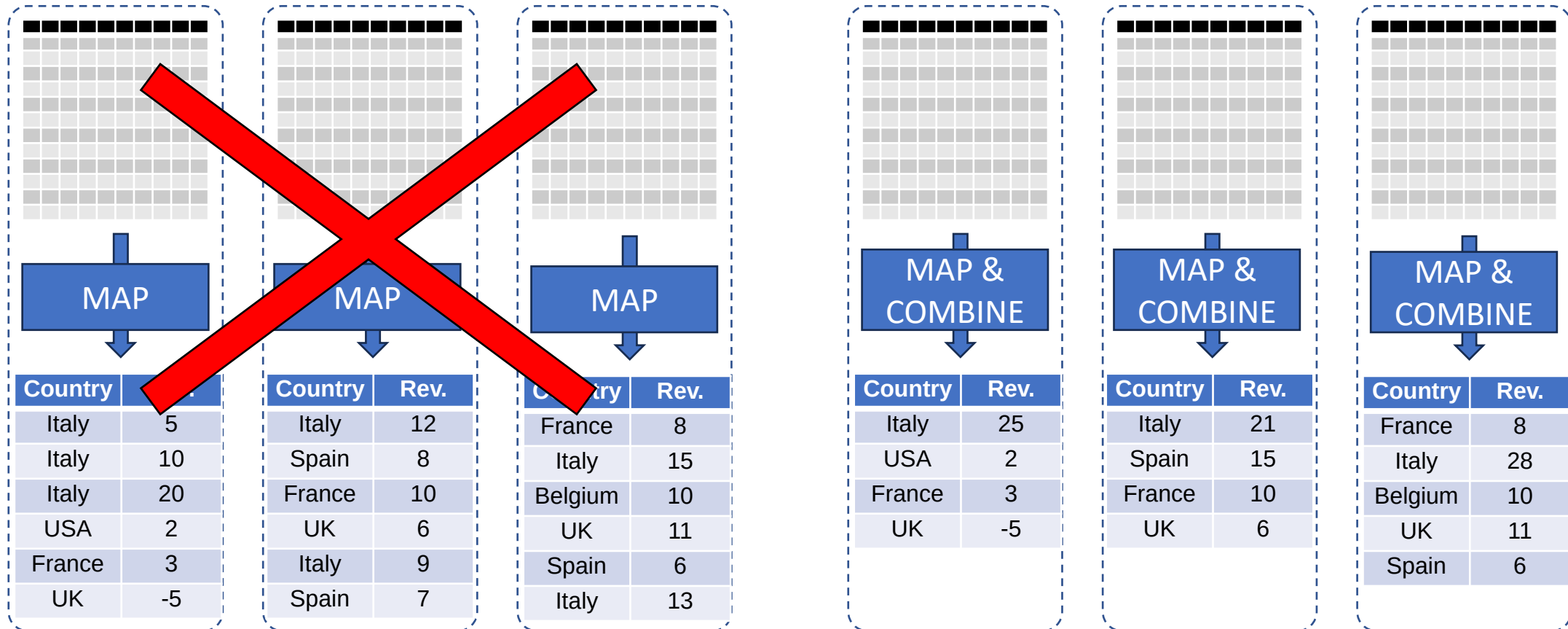
Reduce function: count the number of words in the list and emit a pair of the form:

$$(l, m)$$

Output: l is a length; m is the total number of words of length l in the input documents

MapReduce with combiners

Combining means pre-aggregating data on the Map side



MapReduce process with combiners

When the Reduce function is associative and commutative, we can **push some of what the reducers do to the Map tasks**

In this case we also apply a *Combiner* to the Map function

Grouping is still necessary!

- Values belonging to the same key are (most probably) distributed across multiple splits

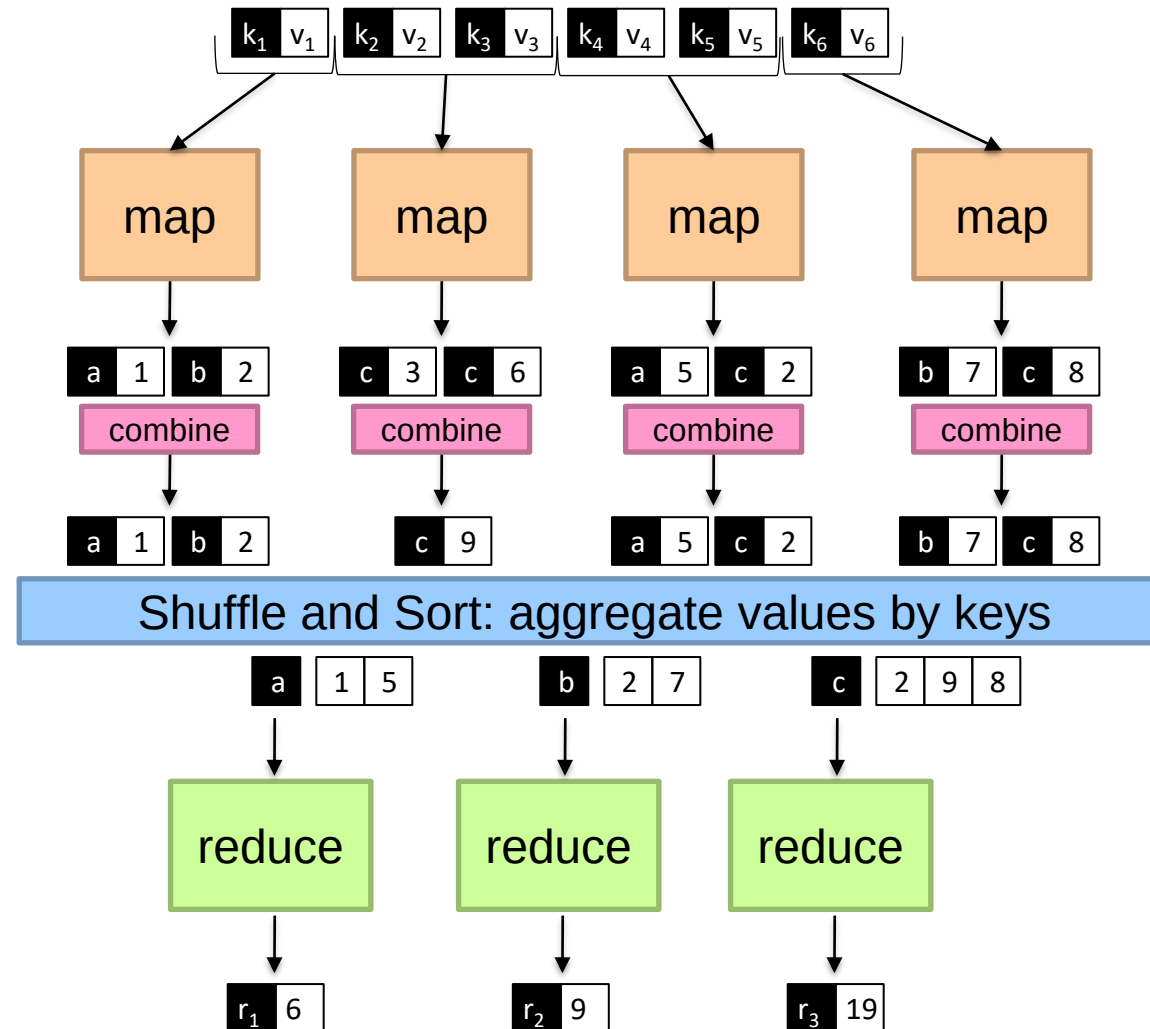
Advantages:

- it reduces the amount of intermediate data
- it reduces the network traffic

The combiner function must be **associative and commutative**

- Associativity: $f(f(a,b),c) = f(a,f(b,c))$ Commutativity: $f(a,b) = f(b,a)$

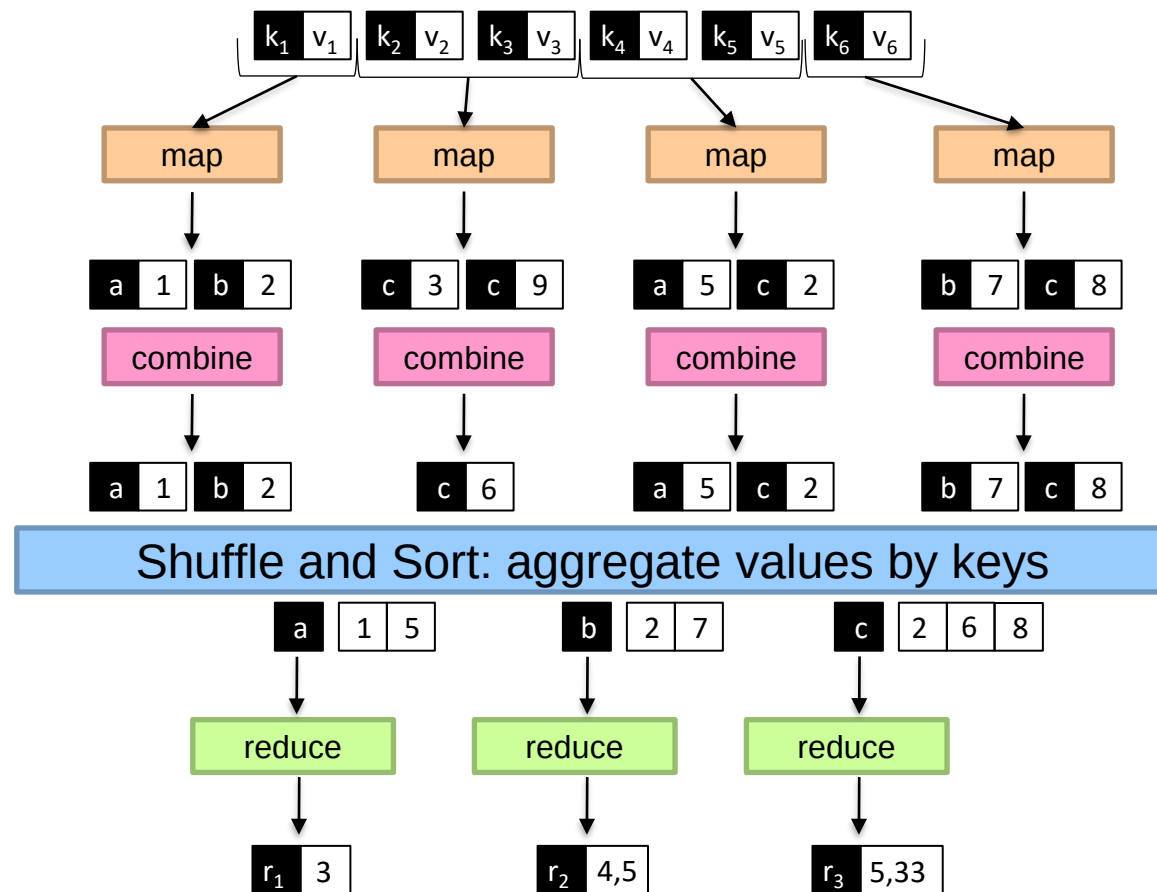
MapReduce process with combiners



Non-associativity

$$f(f(a,b),c) \neq f(a,f(b,c))$$

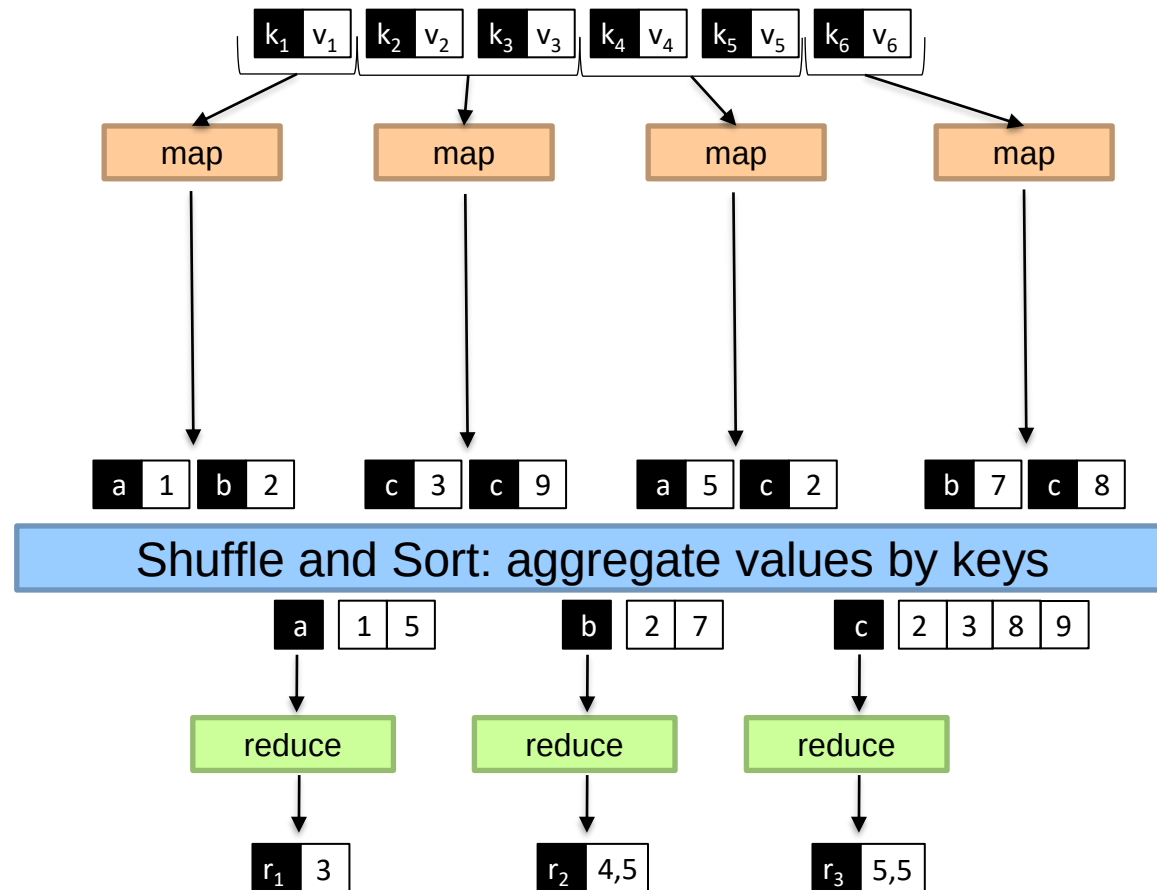
Example:
average
with combiner



Non-associativity

$$f(f(a,b),c) \neq f(a,f(b,c))$$

Example:
average
without combiner

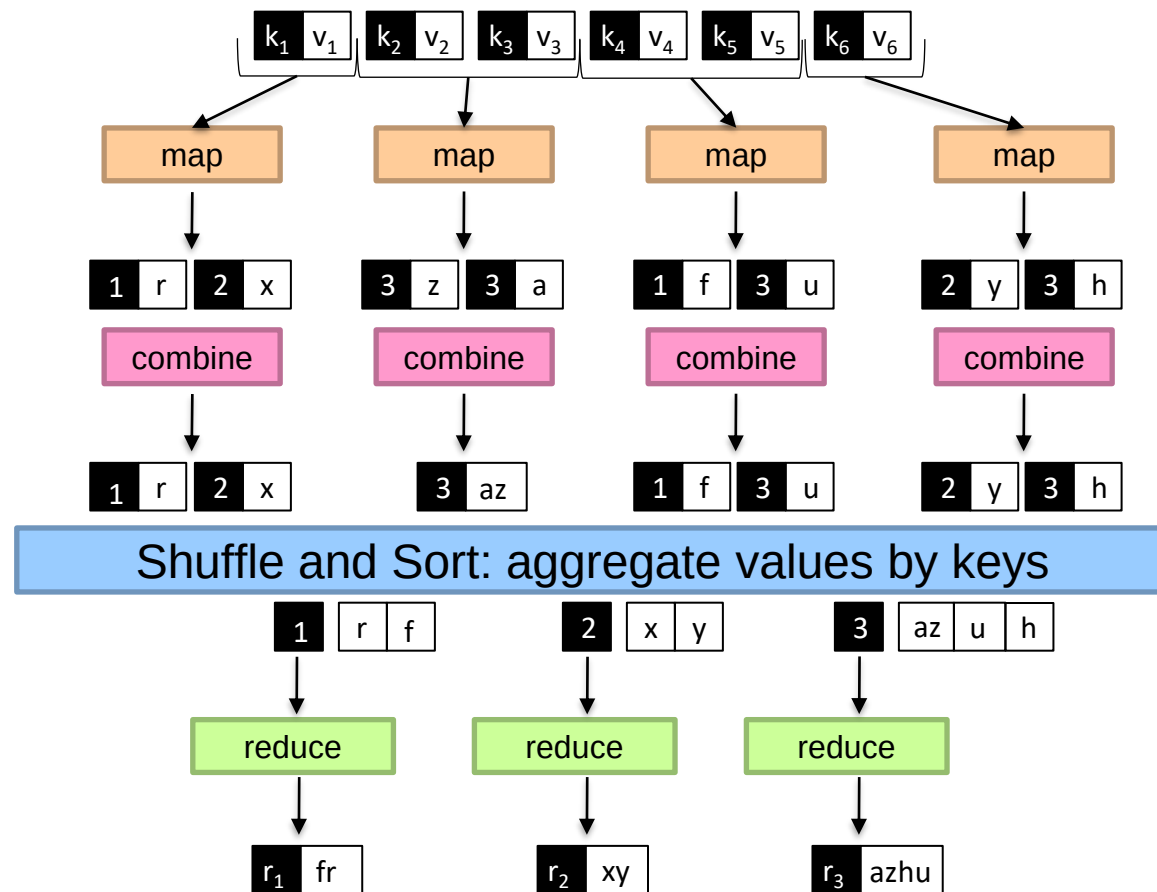


Non-commutativity

$$f(a,b) \neq f(b,a)$$

Example:

concatenation of
alphabetically-ordered
values
with combiner



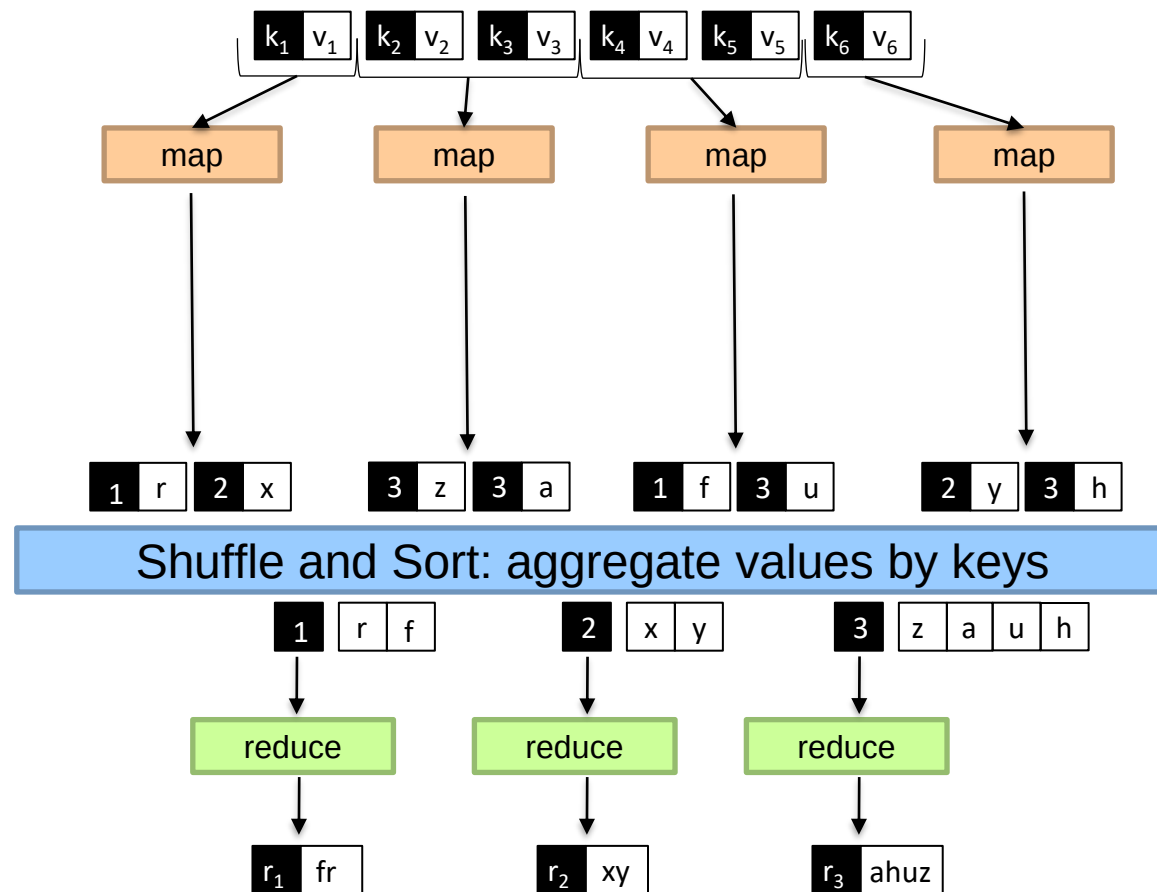
Non-commutativity

$$f(a,b) \neq f(b,a)$$

Example:

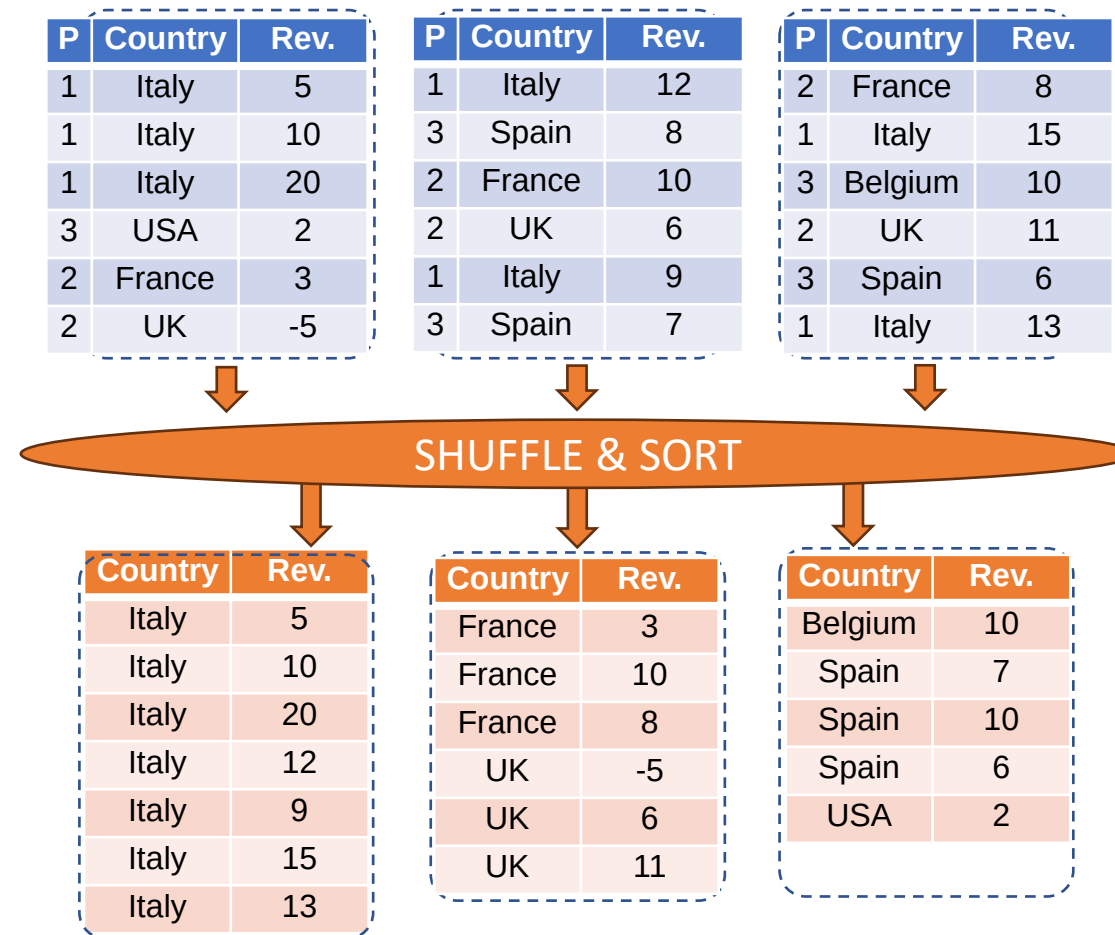
concatenation of
alphabetically-ordered
values

without combiner



MapReduce: partitioning map output

Partitioning map output means assigning to each key-value pair the ID of the reducer that will manage that key



MapReduce: partitioning map output

Maps

- Partitioning depends on the input splits
- One map task per input split

Reducers

- Data is shuffled and according to a partitioning function
 - Decides, for each key, which reducers it goes to
- The number of reducers is tuneable
 - Can be based on the number of nodes and available resources
 - Can be defined by the user
- **The keyspace of the intermediate key-value pairs is evenly distributed over the reducers with a hash function**
 - **Not done in the previous example**

MapReduce: partitioning map output

How does it work?

- Let p be the number of Reduce tasks
 - Either specified by the user or calculated by the system
- The partitioner adopts a hash function that translates a key to a number from 0 to $p - 1$
 - Different keys may end up in the same partition
- Each output key-value pair is stored in one of p files (i.e., partitions)
 - Each file is destined for one of the Reduce tasks
- The reduce task collects from every map task the partitions with the same hash value

MapReduce: partitioning map output

The simplest partitioner assigns approximately the same number of keys to each reducer

- But a partitioner only considers the key and ignores the number of values

An imbalance in the amount of data associated with each key is relatively common in many text processing applications

- In texts, the frequency of any word is inversely proportional to its rank in the frequency table
- The most frequent word will occur approximately twice as often as the second most frequent word, three times as often as the third most frequent word, etc.

The partitioning can be controlled by a user-defined partitioning function

- However, the default partitioner normally works well

MapReduce: number of reduce tasks

A trivial solution: **only one, global Reduce task**

- An obvious bottleneck for a large job

A reasonable solution: **one task per-CPU (many keys per task)**

- Different tasks may take significantly different times, usually because of the non-uniform distribution of values within keys (skewness)

A more reasonable solution: **many tasks per-CPU (less keys per task)**

- Better capable to mitigate skewness problems
 - Time-consuming tasks occupy a CPU fully
 - Many quicker tasks run sequentially in the same CPU
- However, there is overhead associated with each task

Choosing the number of reducers for a job is more of an art than a science

- Rule of thumb: a task should run for at least 100-200 ms and process at least some records; in large jobs, it is ok to run for ~5 minutes and produce ~1 DFS block's worth of output

MapReduce: the complete picture

Programmers specify two functions:

map $(k1, v1) \rightarrow \text{list}(k2, v2)$

reduce $(k2, \text{list}(v2)) \rightarrow \text{list}(k3, v3)$

- All values with the same key are reduced together

Usually, programmers also specify:

combine $(k2, \text{list}(v2)) \rightarrow \text{list}(k3, v3)$

- Mini-reducers that run after the map phase
- Used as an optimization to reduce network traffic

partition $(k2, \text{number_of_partitions}) \rightarrow \text{partition_for_k2}$

- Divides up key space for parallel reduce operations

The execution framework handles everything else...

MapReduce algorithms

Algorithms for MapReduce

MapReduce is a paradigm, not a tool

- You must fit your solution into the framework of map and reduce
- In some situations, it might be challenging
- Translating machine learning and data mining algorithms to the MapReduce paradigm is not trivial

Note that sometimes we may need **multiple map / reduce stages** – chains of maps and reduces

Filtering algorithms

Goal: find lines/files/tuples with a particular characteristic

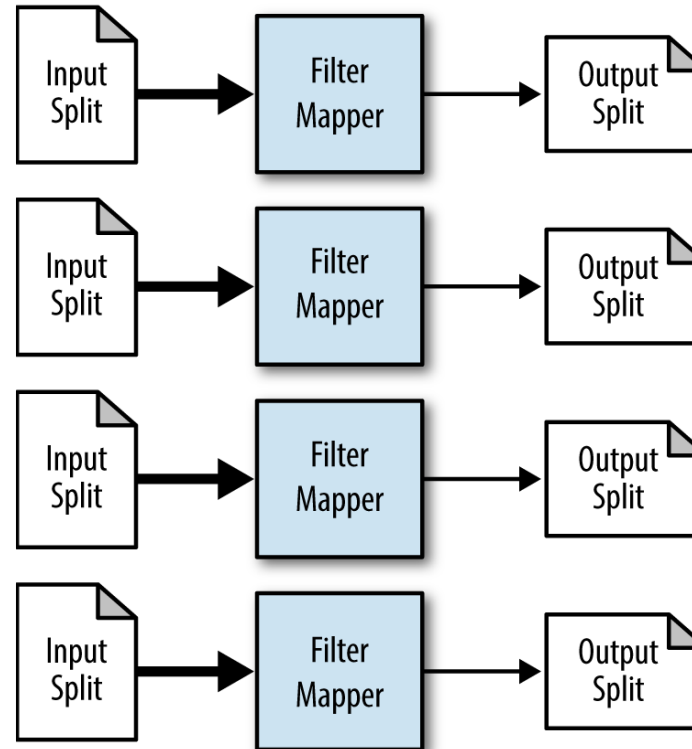
Examples:

- Retrieve web logs for requests to a certain domain (or from a certain IP)
- Retrieve a sample of the dataset
- Find the 10 customers with the highest reputation
 - Given that the reputation is already stored in the data

General pattern

- `map (key, record) → if (criteria is met) then emit(key, record)`
- `reduce (key, record) → if (criteria is met) then emit(key, record)`
 - Reduce may even be omitted

Filtering pattern



Summarization algorithms

Goal: compute the maximum/sum/average/... over a set of values

Examples:

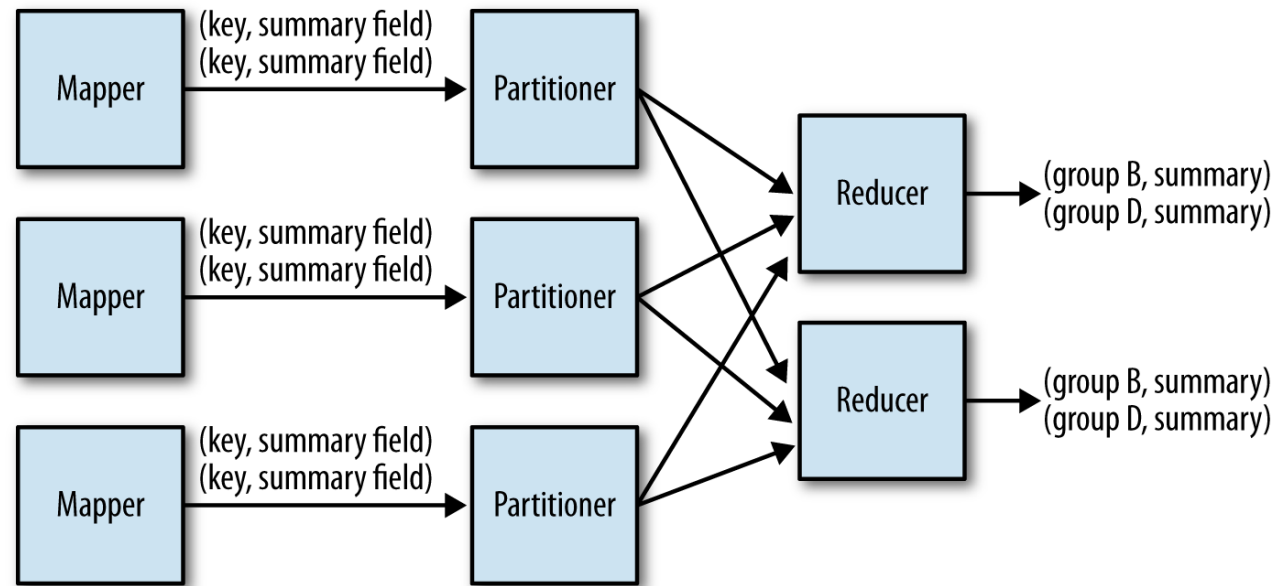
- Count the number of requests to each subdomain of *.csr.unibo.it
- Find the most popular domain (!)
- Build an inverted index of words in documents

General pattern:

- map (key, record) → foreach group_criteria in record :
emit (group_criteria, value)
- reduce (group_criteria, values) → emit (group_criteria, agg(values))

A combiner may be used to perform a pre-aggregation

Summarization pattern



Join

Goal: combine different inputs on some shared values

Example:

- Given a list of professors (with the courses they teach) and a list of students (with the courses they follow), find all triples <professor, course, student> where the course is in common

General pattern:

- `map (key, record) → if (dataset1)`
 `then emit (sharedKey, (flag1, record) )`
 `else emit (sharedKey, (flag2, record) )`
- `reduce (sharedKey, records) → for r1 in records with flag1 :`
 `for r2 in records with flag2 :`
 `emit (someKey, <r1, sharedKey, r2> )`

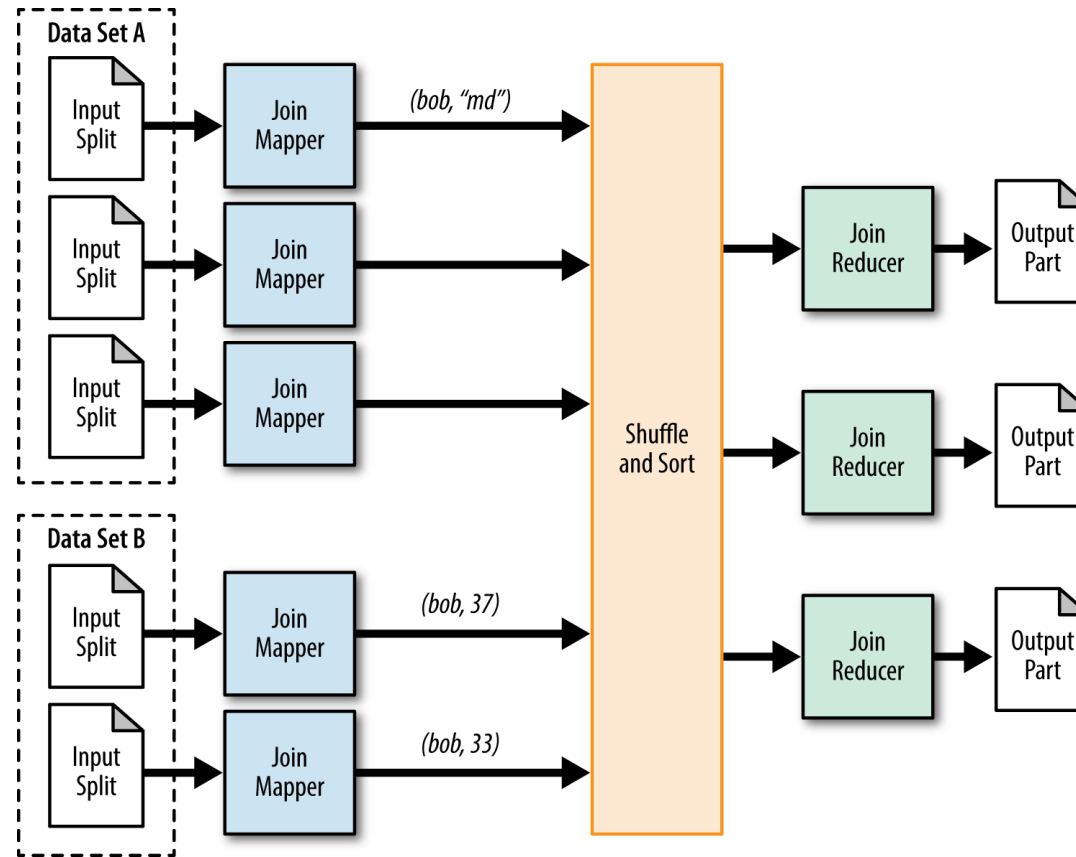
Join

Example:

- Prof(Name, Course) = {(Enrico, BigData), (Matteo, BigData)}
- Stud(Name, Course) = {(Mario, BigData), (Lucia, DataMining)}
- Prof JOIN Stud ON (Prof.Course=Stud.Course)

- MAP: {(BigData, (Prof, Enrico)), (BigData, (Prof, Matteo)),
(BigData, (Stud, Mario)), (DataMining, (Stud, Lucia))}
- SHUFFLE AND SORT: {(BigData, [(Prof, Enrico), (Prof, Matteo), (Stud, Mario)])
(DataMining, [(Stud, Lucia)])}
- REDUCE: {(k1, (Enrico, BigData, Mario)), (k2, (Matteo, BigData, Mario))}

Join pattern



Sort

Goal: sort input

Examples:

- Return all the domains indexed by Google and the number of pages in each, ordered by the number of pages

The programming model (map, then reduce) does not support this per se

- But the implementations do: the shuffle stage performs grouping and ordering!

General pattern:

- `map (key, record) → emit (sortKey, record)`
- `reduce (sortKey, records) → emit (sortKey, records[1]), ...`

The Map and the Reduce do nothing

- With 1 reducer, we get sorted output
- With many reducers, we get partly sorted output (unless: `TotalOrderPartitioner`)

Two stage MapReduce

As map-reduce calculations get more complex, it's useful to break them down into stages

- The output of the first stage serves as input to the next one
- The **same output** may be useful for **different subsequent stages**
- The output can be stored in the DFS, forming a **materialized view**

Early stages of map-reduce operations often represent the heaviest amount of data access, so building and saving them once as a basis for many downstream uses saves a lot of work!

Two stage MapReduce

Example: compare the sales of products for each month in 2011 to 2010

First stage: produce records showing the **sales** for a single **product** in a single **month** of a single **year**

- map (key, sales) → for s in sales :
 emit(concat(s.month, s.year, s.product), s.qty)
- reduce (keyMYP, quantities) → emit (keyMYP, sum(quantities))

Second stage: produce records showing the **increase of sales** for a single **product** in a single **month** by **comparing 2011 sales with 2010 ones**

- map (keyMYP, totalQty) →
emit(concat(s.month, s.product),(s.year, s.totalQty))
- reduce (keyMP, totals) → emit (keyMP, (totals[2011]/totals[2010]))